

SYSTEM DESIGN RATIONALE

University Course Registration, Timetable & Attendance Management
System

CO2203 – Object Oriented Programming
Architecture & Design Specification

Group Name :UniVerse

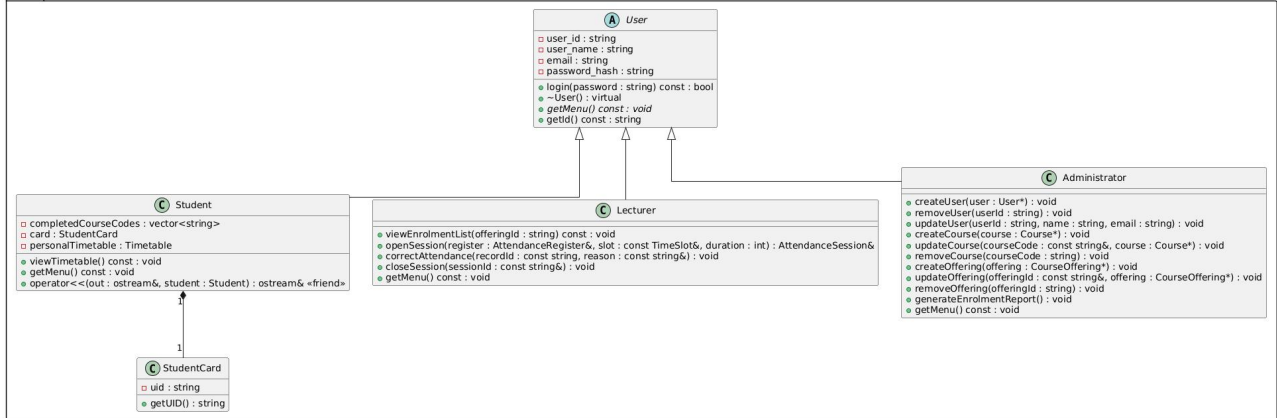
Group Members : R.N. KESHAN : 24/ENG/119

M.P.D.M. PITIYEGEDARA : 24/ENG/116

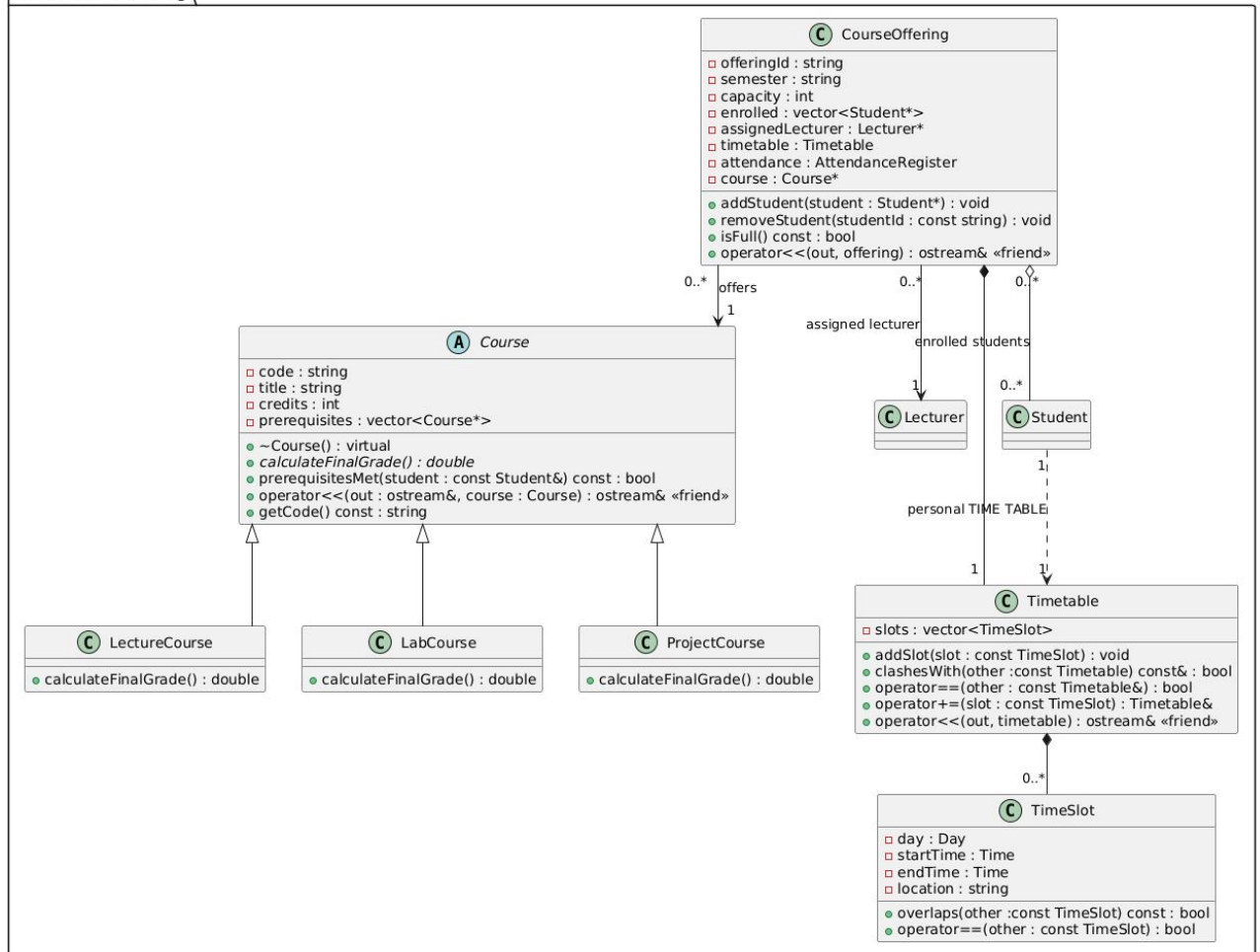
W.K.P.M. THILAKARATHNE : 24/ENG/152

UML Design

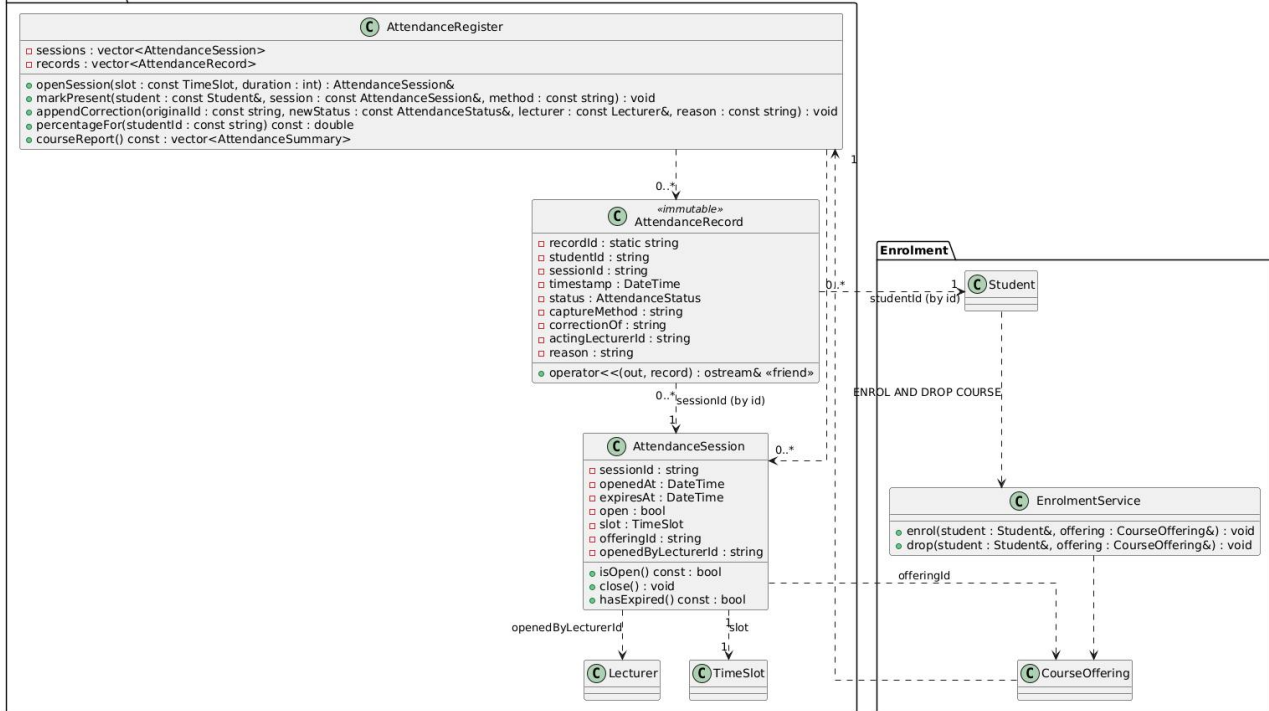
Users



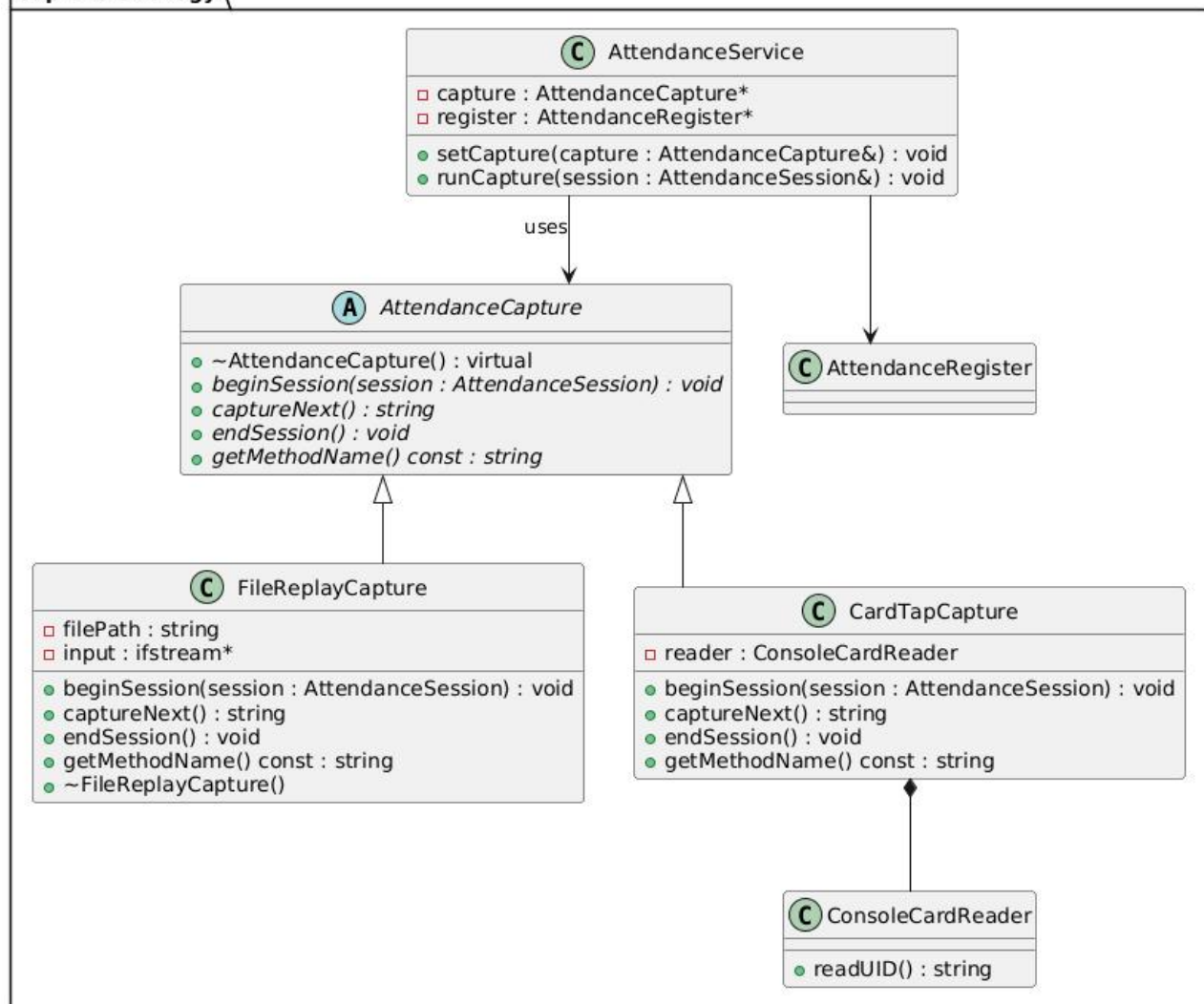
Courses & Scheduling



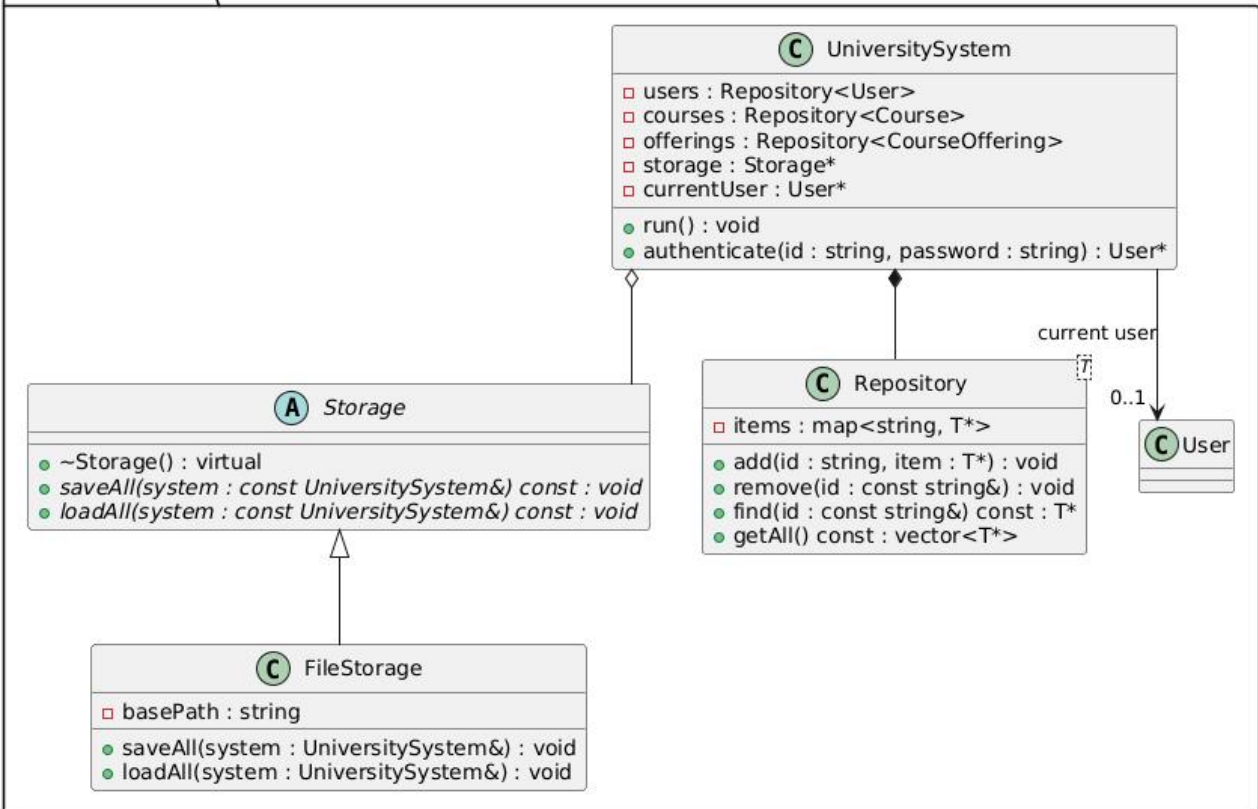
Attendance Core



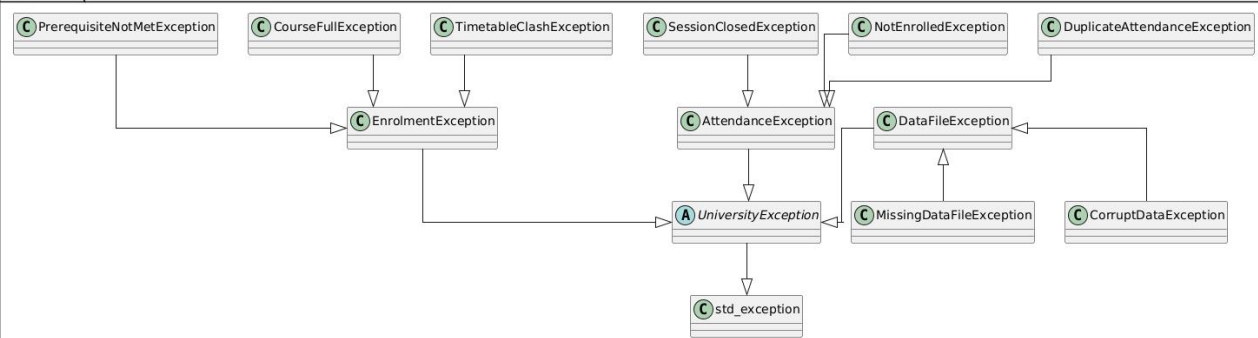
Capture Strategy



Persistence & UI



Exceptions



1. Introduction

This system is developed for essential tasks such as university course registration, releasing timetables through it and marking attendance through it. So, this system basically developed based on C++ with object-oriented programming concepts. This system provides few roles such as Student, Lecturer and administrator. This can provide functionalities for management of timetables, attendance, registrations, reporting and store system data as well.

In this system we have used concepts such as abstraction, polymorphism, encapsulation, inheritance, aggregation, templates, operator overloading and file handling to create a realistic and professional for digital university.

The system contains six focus areas. It includes users, courses and scheduling, attendance, capture strategy and persistence and user interface.

2. Class Hierarchy

2.1. User Hierarchy

There is an abstract user base class that has three derived classes as Student, Lecturer and Administrator for the user hierarchy. There are few attributes that are same for all of them such as user id, username, email, password and login functionality. They are also unique operations according to their role as well.

By using getMenu() function polymorphism can be implemented through pure virtual. Each derived class performs their own tasks of this function. It helps with the appropriate menu based on their role. After verifying the credential for the login system this system will interact with User*pointer and calls getMenu(). Then system can automatically recognize the user role as well.

Then the hierarchy can also increase the reusability of the code and minimize the complexity of it. Making edits such as adding and removing also makes it easier with this system.

2.2. Course Hierarchy

There is an abstract user base class that has three derived classes such as LectureCourse, LabCourse and ProjectCourse for the course hierarchy. This also includes common attributes such as course code, title, credits. Each course can calculate their final marks as well.

Polymorphism is achieved through the pure virtual function calculateFinalGrade(). Each derived class provides its own implementation based on the course type.

This hierarchy also helps to improve reusability, avoids duplications for same thing for each class. Modifications of the system are also provided here as well.

2.3. Attendance Capture Hierarchy

The AttendanceCapture structure differentiates the attendance input method from the attendance processing logic by establishing a shared interface via the abstract AttendanceCapture class, which is realized by FileReplayCapture and CardTapCapture. AttendanceService connects with the active capture method via a unique_ptr, allowing various attendance capture strategies to be chosen during runtime without altering the attendance management process. The required FileReplayCapture allows for testing with recorded data, whereas CardTapCapture (Simulated Card Tap) tracks attendance by scanning each student's StudentCard, giving the system adaptability and straightforward extensibility.

2.4. Storage Hierarchy

The Storage class is like a plan that has methods to save and load data. These methods are saveAll and loadAll. They make it easy to store and get data. FileStorage does what the Storage class says it should do. This helps the UniversitySystem work, with kinds of storage. It also makes it easy to change how data is stored on without changing how the UniversitySystem works. The Storage class and FileStorage make this possible.

2.5. Exception Hierarchy

The system uses a list of exceptions. `UniversityException` is the class. It has exceptions like `ValidationException` and `EnrollmentException` and `AttendanceException`. These exceptions help the system know what kind of error is happening. This way the system can handle errors in a way. It can catch all the exceptions with the main class `UniversityException`. It can also give different error messages for different types of errors. This makes it easy to know what went wrong. The system can handle all the `UniversityException` errors in one place. It can still give specific error messages, for `ValidationException` errors or `EnrollmentException` errors or `AttendanceException` errors.

3. Composition vs. Aggregation

Composition implies ownership and shared lifetimes, whereas aggregation implies a looser connection between two independently existing objects. In our design, composition is applied whenever `Student` has `StudentCard` and `Timetable`, `CourseOffering` has `Timetable` and `AttendanceRegister`, `Timetable` has its `TimeSlot` objects, `AttendanceRegister` has its `AttendanceSession` and `AttendanceRecord` objects, `AttendanceSession` has its `TimeSlot`, `CardTapCapture` has `ConsoleCardReader`, and `UniversitySystem` has its repositories. The contained objects are kept as member objects and are destroyed along with their owners. In contrast, aggregation is applied between `CourseOffering` and `Student` because the former contains only non-owning pointers to students; students may live independently, be enrolled in more than one course offering, and do not get deleted when a course offering is deleted. Moreover, aggregation is applied between `UniversitySystem` and `Storage` because the former just refers to `Storage`, which is externally created and is not managed by the `UniversitySystem`.

4. Mandatory OOP Requirements

OOP Requirement	Where in the UML	Why it is Used
Classes and Objects	All classes (<code>User</code> , <code>Course</code> , <code>Attendance Session</code> , <code>Attendance Register</code> , etc.)	For representing real-world entities of the university as objects with its own data and behavior.
Encapsulation	Private data member(-)s with public member functions in all classes	Prevents direct access to internal data and provides controlled access via member functions.
Inheritance	<code>User</code> -> <code>Student</code> , <code>Lecturer</code> , <code>Administrator</code> ; <code>Course</code> -> <code>LectureCourse</code> , <code>LabCourse</code> , <code>ProjectCourse</code> ; <code>AttendanceCapture</code> -> <code>FileReplayCapture</code> , <code>CardTapCapture</code> ; <code>Storage</code> -> <code>FileStorage</code> ; and the exception hierarchy.	Prevents code repetition and provides specialized behavior in derived classes.
Polymorphism	Pure virtual functions such as <code>User::getMenu()</code> , <code>Course::calculateFinalGrade()</code> , <code>AttendanceCapture</code> methods, and <code>Storage::saveAll()</code> / <code>loadAll()</code>	It allows the system to access derived classes implementations via base class pointers or references.
Abstract Classes	<code>User</code> , <code>Course</code> , <code>Attendance Capture</code> , <code>Storage</code> , <code>University Exception</code>	It does not allow creating objects directly and provides interfaces for the derived classes.
Composition	<code>Student</code> *-- <code>StudentCard</code> ; <code>CourseOffering</code> *-- <code>Timetable</code> ; <code>Timetable</code> *-- <code>TimeSlot</code> ;	It represents strong containment where the contained objects rely on the lifetime of the containing object.
Aggregation	<code>Student</code> — <code>Course Offering</code> ,	It represents associations where the objects do not depend on

		each other's lifetimes.
Templates	Repository<T>	It provides a reusable generic repository, which will allow storing different object types without replicating code.
Exception Handling	University Exception hierarchy and throw/catch in validation and enrolment operations	Handles run-time exceptions gracefully and gives informative error messages.
File Handling	FileStorage, Storage::saveAll(), Storage::loadAll(), and FileReplayCapture::input.	It allows saving/loading the state of the application for persistent storage.
Operator Overloading	operator<< in Student, Course, CourseOffering, Timetable, and AttendanceRecord; operator== in TimeSlot and Timetable; Timetable::operator+=.	Helps in creating formatted output of the objects.
STL Containers & Smart Pointers	vector, map	It helps manage collections of objects efficiently and automatically handles memory.

5. Selected Interactive Attendance Capture Method

The system optimizes Option B – Simulated Card Tap as the interactive attendance capture method. This approach was taken since it is close to a real university card-based attendance system and it is simple to implement in a console application. Every undergraduate receives an assigned unique Student Card, and the attendance of them are read through the card UID using ConsoleCardReader and records attendance through CardTapCapture providing a realistic, modular, and easily extensible solution.

6. Mapping the Design to Three Member Roles

Member	Main Responsibility	Related Classes/Modules
Member 3	User and course management	User, Student, Lecturer, Administrator, Course, Lecture Course, Lab Course, and Project Course
Member 2	Enrolment and timetable management	Course Offering, Time Slot, Enrolment operations, prerequisite checking, and timetable conflict handling
Member 1	Attendance and data persistence	Attendance Session, Attendance Register, Attendance Record, Attendance Capture, CardTapCapture, FileReplayCapture, Storage, and File Storage

The system is partitioned so that every individual is accountable for an independent functional module with a clearly defined public interfaces. This ensures reduced development conflicts, allows individuals to work independently, and supports easier integration and testing of the entire system.

Interface Agreement

Exposed by (Class and Owner)	Public Function	Used by	Purpose
User (Member 3)	getId() const	EnrolmentService::enrol()/drop() (Member 2) AttendanceRegister::markPresent() /appendCorrection() (Member 1)	Providing Student and Lecturer IDs without exposing private data
StudentCard (Member 3)	getUID() const	CardTapCapture::captureNext() (Member 1)	Matching a tapped card UID with the correct student
Course (Member 3)	prerequisitesMet(const Student&) const	EnrolmentService::enrol() (Member 2)	Checking prerequisites before enrolment
Course (Member 3)	getCode() const	EnrolmentService::enrol() (Member 2)	Obtaining the course code for enrolment and prerequisite checks
EnrolmentService (Member 2)	enrol(Student&, CourseOffering&)	Student::getMenu() enrolment action (Member 3)	Enrolling a student after all required checks pass
EnrolmentService (Member 2)	drop(Student&, CourseOffering&)	Student::getMenu() drop action (Member 3)	Removing a student from a course offering
CourseOffering (Member 2)	isFull() const	Student/Administrator menu logic (Member 3)	Displaying whether an offering has available capacity
TimeSlot (Member 2)	overlaps(const TimeSlot&) const	AttendanceRegister::openSession() (Member 1)	Preventing overlapping attendance sessions
Timetable (Member 2)	operator<<(ostream&, const Timetable&)	Student::viewTimetable() (Member 3)	Displaying a student's personal timetable
AttendanceRegister (Member 1)	openSession(const TimeSlot&, int)	Lecturer::openSession() (Member 3)	Creating a new attendance session for a lecturer
AttendanceRegister (Member 1)	appendCorrection(origi nalId, status, lecturer, reason)	Lecturer::correctAttendance() (Member 3)	Adding an auditable attendance correction
AttendanceSession (Member 1)	close()	Lecturer::closeSession() (Member 3)	Stopping further attendance capture for a session
AttendanceRegister (Member 1)	courseReport() const	Administrator::generateEnrolment Report() (Member 3)	Providing attendance summaries for administrative reporting
AttendanceService (Member 1)	setCapture(Attendance Capture&)	Lecturer attendance workflow (Member 3)	Selecting file-replay or card-tap attendance capture
AttendanceService (Member 1)	runCapture(Attendance Session&)	Lecturer attendance workflow (Member 3)	Running the selected capture method for an open session
Storage (Member 1)	saveAll(const UniversitySystem&)	UniversitySystem::run() (Owner to be assigned)	Saving application data when required or before exit
Storage (Member 1)	loadAll(UniversitySyst em&)	UniversitySystem::run() (Owner to be assigned)	Loading saved application data during startup